

SISTEM ANTRIAN TIKET BIOSKOP BERBASIS QUEUE DAN LINKED LIST

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Intan Sahara Nurafni J0403251005

Naylatul Fadhillah J0403251033

Faiza Aghnaita Zahra J0403251122

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan Project Based Learning (PBL) mata kuliah Algoritma & Struktur Data yang berjudul "**Sistem Antrian Tiket Bioskop Berbasis Queue dan Linked List**" dengan baik.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas akhir Project Based Learning (PBL) pada mata kuliah Algoritma & Struktur Data. Dalam penyusunannya, penulis menerapkan konsep struktur data, algoritma, serta pemrograman Python untuk membangun sistem antrian tiket bioskop yang mampu mengelola data pembeli secara terstruktur dan tersimpan secara permanen.

Penulis mengucapkan terima kasih kepada **Dr. Shelve Nidya Neyman** selaku dosen pengampu mata kuliah Algoritma & Struktur Data atas bimbingan dan ilmu yang telah diberikan. Penulis juga mengucapkan terima kasih kepada seluruh anggota kelompok yang telah bekerja sama dan memberikan kontribusi dalam penyelesaian proyek ini.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan maupun pengembangan proyek di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat serta menjadi referensi dalam memahami penerapan struktur data dan algoritma pada permasalahan nyata.

Bogor, 22 Juni 2026

Kelompok 13 - Threecute

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I. PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan Proyek.....	5
1.3 Ruang Lingkup.....	5
BAB II. ANALISIS DAN PERANCANGAN	7
2.1 Deskripsi Sistem	7
2.2 Struktur Data yang Digunakan.....	7
2.3 Algoritma yang Digunakan.....	8
2.4 Diagram Alir Program (Flowchart)	10
2.5 Struktur Program.....	10
BAB III. IMPLEMENTASI.....	12
3.1 Tampilan Menu Utama	12
3.2 Fitur Create	12
3.3 Fitur Read.....	13
3.4 Fitur Update	14
3.5 Fitur Delete	14
3.6 Fitur Searching.....	15
3.7 Fitur Sorting	16
3.8 Penyimpanan Data (File Handling)	16
3.9 Validasi Input dan Error Handling.....	19
BAB IV. PENGUJIAN.....	21
4.1 Skenario Pengujian	21
4.2 Kelebihan Sistem	22
4.3 Keterbatasan Sistem.....	23
BAB V. KENDALA DAN SOLUSI.....	24
BAB VI. PEMBAGIAN TUGAS	26

BAB VII. KESIMPULAN DAN SARAN.....	27
Kesimpulan	27
Saran Pengembangan	27
DAFTAR PUSTAKA.....	29
LAMPIRAN.....	30
Lampiran A. Link Github	30
Lampiran B. Screenshot Program	30
Lampiran C. Struktur Folder Proyek	33
Lampiran D. Commit History Github	33
Lampiran E. Source Code Utama	34

BAB I. PENDAHULUAN

1.1 Latar Belakang

Proyek ini dibuat untuk membantu pemilik bioskop dalam mengelola antrian pembelian tiket bioskop secara terstruktur. Sistem menggunakan Queue berbasis Linked List agar pembeli dilayani sesuai urutan kedatangan atau FIFO. Data akan disimpan pada file TXT sehingga tetap tersedia apabila program ingin dijalankan kembali.

1.2 Tujuan Proyek

Adapun tujuan proyek ini dibuat ialah:

1. Membangun aplikasi sistem antrian tiket bioskop berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python untuk membantu pengelolaan data pembeli secara sederhana, terstruktur, dan mudah digunakan.
2. Mengimplementasikan struktur data Queue dan Linked List dalam proses pengelolaan antrian sehingga pembeli dilayani sesuai urutan kedatangan (*First In First Out/FIFO*) serta memudahkan penyimpanan dan pengelolaan data antrian.
3. Menerapkan algoritma Sequential Search dan Merge Sort untuk mendukung proses pencarian data pembeli berdasarkan nama atau nomor kursi serta pengurutan data nomor kursi secara lebih terstruktur.
4. Menerapkan konsep file handling dan validasi input agar data antrian serta informasi ketersediaan kursi tersimpan secara permanen pada file TXT, sekaligus meminimalkan kesalahan input pengguna selama menjalankan program.

1.3 Ruang Lingkup

Ruanglingkup proyek ini meliputi:

- Program dikembangkan menggunakan bahasa pemrograman Python berbasis Command Line Interface (CLI).
- Sistem digunakan untuk mengelola proses antrian pembelian tiket bioskop.
- Struktur data yang digunakan adalah Queue berbasis Linked List untuk mengatur urutan pelayanan berdasarkan prinsip First In First Out (FIFO).

- Penyimpanan data dilakukan menggunakan file TXT sehingga data antrian dan jumlah kursi tetap tersimpan meskipun program ditutup.
- Sistem menyediakan fitur CRUD (Create, Read, Update, Delete) yang meliputi penambahan data antrian, menampilkan data antrian, mengubah nomor kursi, dan m
- elayani atau menghapus antrian terdepan.
- Sistem menyediakan fitur Searching berdasarkan nama pembeli atau nomor kursi menggunakan algoritma Sequential Search.
- Sistem menyediakan fitur Sorting berdasarkan nomor kursi menggunakan algoritma Merge Sort.
- Sistem dapat menampilkan antrian terdepan (Front Queue), jumlah antrian, dan informasi sisa kursi yang masih tersedia.
- Sistem menerapkan validasi input untuk memastikan nama pembeli dan nomor kursi yang dimasukkan sesuai ketentuan serta mencegah penggunaan nomor kursi yang telah dipilih sebelumnya.

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem Antrian Tiket Bioskop merupakan aplikasi berbasis Command Line Interface (CLI) yang dikembangkan menggunakan bahasa pemrograman Python untuk membantu pengelolaan proses antrian pembelian tiket bioskop secara terstruktur. Sistem ini menerapkan struktur data Queue berbasis Linked List sehingga setiap pembeli akan dilayani sesuai urutan kedatangannya berdasarkan prinsip *First In First Out* (FIFO).

Selain itu, pengguna dapat mengelola data antrian melalui fitur Create, Read, Update, Delete (CRUD), melakukan pencarian data berdasarkan nama atau nomor kursi menggunakan Sequential Search, serta mengurutkan data nomor kursi menggunakan Merge Sort. Selain itu, sistem dilengkapi dengan validasi input untuk mengurangi kesalahan saat pengisian data dan memanfaatkan file handling menggunakan file TXT agar data antrian serta informasi ketersediaan kursi tetap tersimpan secara permanen meskipun program ditutup dan dijalankan kembali.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
Linked List	Digunakan sebagai struktur data utama untuk menyimpan seluruh data antrian pembeli tiket bioskop. Setiap node menyimpan informasi berupa nama pembeli, nomor kursi, dan pointer (<i>next</i>) yang menghubungkan ke node berikutnya. Struktur ini memungkinkan proses penambahan, penghapusan, pencarian, dan penelusuran data dilakukan secara efisien tanpa menggunakan array berukuran tetap.
Queue	Digunakan untuk mengatur urutan pelayanan pembeli berdasarkan prinsip First In First Out (FIFO). Queue

diimplementasikan menggunakan pointer front dan rear, sehingga proses enqueue (menambah antrian) dilakukan pada bagian belakang (*rear*), sedangkan proses **dequeue** (melayani antrian) dilakukan pada bagian depan (*front*). Implementasi ini memastikan pembeli yang datang lebih dahulu akan dilayani lebih dahulu.

Alasan Pemilihan Struktur Data

Linked List dipilih karena mampu menyimpan data antrian secara dinamis tanpa perlu menentukan kapasitas penyimpanan di awal. Selain itu, struktur ini memudahkan proses penambahan node baru di akhir antrian, penghapusan node di awal antrian, serta penelusuran data pembeli secara efisien sesuai kebutuhan sistem.

Kemudian, Queue dipilih karena menerapkan prinsip First In First Out (FIFO), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Prinsip ini sangat sesuai dengan sistem antrian tiket bioskop karena setiap pembeli akan dilayani berdasarkan urutan kedatangannya, sehingga proses pelayanan menjadi lebih adil, teratur, dan mudah dikelola.

2.3 Algoritma yang Digunakan

Searching

- Sequential Search

Alasan penggunaan:

Sequential Search digunakan karena data antrian disimpan dalam struktur Linked List, sehingga proses pencarian dilakukan dengan menelusuri setiap node secara berurutan dari antrian paling depan hingga akhir. Algoritma ini digunakan untuk mencari data pembeli berdasarkan nama atau nomor kursi. Sequential Search dipilih karena implementasinya sederhana, sesuai dengan ukuran data yang tidak terlalu besar, serta tidak memerlukan data dalam keadaan terurut.

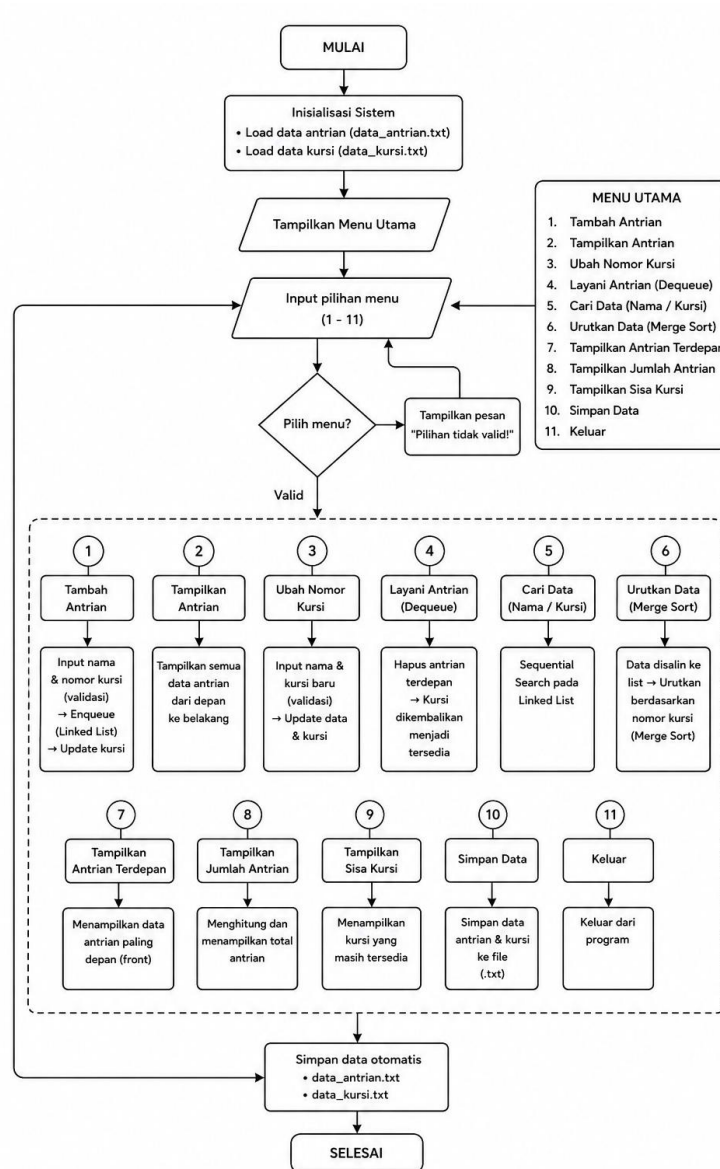
Sorting

- Merge Sort

Alasan penggunaan:

Merge Sort digunakan untuk mengurutkan data berdasarkan nomor kursi sebelum ditampilkan kepada pengguna. Algoritma ini dipilih karena memiliki kompleksitas waktu $O(n \log n)$ sehingga lebih efisien dibandingkan algoritma pengurutan sederhana seperti Bubble Sort atau Insertion Sort, terutama ketika jumlah data semakin banyak. Selain itu, Merge Sort menghasilkan data yang telah terurut tanpa mengubah struktur Linked List yang digunakan sebagai penyimpanan utama, karena proses pengurutan dilakukan pada data yang terlebih dahulu dikonversi ke dalam list.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Struktur program pada proyek ini terdiri dari beberapa file yang memiliki fungsi masing-masing, yaitu:

`pjbl.py`

`data_antrian.txt`

`data_kursi.txt`

Penjelasan:

- `pjbl.py` merupakan file utama yang berisi seluruh kode program, meliputi implementasi kelas `Node`, kelas `QueueBioskop`, menu utama, fitur CRUD, algoritma Sequential Search, Merge Sort, validasi input, serta file handling.
- `data_antrian.txt` digunakan untuk menyimpan data seluruh antrian pembeli secara permanen. Setiap data terdiri atas nama pembeli dan nomor kursi yang dipisahkan menggunakan karakter.
- `data_kursi.txt` digunakan untuk menyimpan informasi jumlah kursi yang masih tersedia sehingga data tidak akan kembali ke kondisi awal ketika program dijalankan ulang.

BAB III. IMPLEMENTASI

3.1 Tampilan Menu Utama

```
=== SISTEM ANTRIAN TIKET BIOSKOP ===  
1. Tambah Antrian  
2. Tampilkan Antrian  
3. Update Nomor Kursi  
4. Layani Antrian  
5. Cari Data  
6. Sorting Nomor Kursi  
7. Lihat Antrian Depan  
8. Jumlah Antrian  
9. Info Kursi  
10. Keluar  
Pilih menu (1-10) :
```

3.2 Fitur Create

```
Pilih menu (1-10) : 1  
Masukkan nama pembeli : Gita  
Masukkan nomor kursi : H9  
Gita berhasil masuk antrian.
```

Fitur Create digunakan untuk menambahkan data pembeli baru ke dalam antrian. Pengguna diminta memasukkan nama pembeli dan nomor kursi yang diinginkan. Sistem akan melakukan validasi terhadap nama dan format nomor kursi, kemudian memastikan kursi belum digunakan oleh pembeli lain. Apabila seluruh validasi terpenuhi, data akan ditambahkan ke antrian menggunakan proses enqueue, jumlah kursi yang tersedia akan berkurang, dan data akan disimpan secara otomatis ke dalam file `data_antrian.txt` serta `data_kursi.txt`.

3.3 Fitur Read

```
Pilih menu (1-10) : 2

=== DAFTAR ANTRIAN BIOSKOP ===
1. Cantikya - Kursi C5
2. Andi - Kursi F1
3. Rina - Kursi B9
4. Dafa - Kursi J4
5. Aulia - Kursi D8
6. Rafi - Kursi E2
7. Zahra - Kursi G4
8. Nabila - Kursi G6
9. Fikri - Kursi I10
10. Cindy - Kursi H8
11. Bagas - Kursi C9
12. Putri - Kursi J3
13. andre - Kursi E3
14. intan - Kursi A5
15. lulu - Kursi B8
16. fafa - Kursi F2
17. Gita - Kursi H9
```

Fitur Read digunakan untuk menampilkan seluruh data pembeli yang berada di dalam antrian. Data ditampilkan sesuai urutan kedatangan pembeli dengan melakukan penelusuran pada Linked List mulai dari node paling depan (*front*) hingga node terakhir (*rear*).

3.4 Fitur Update

```
Pilih menu (1-10) : 3

=== DAFTAR ANTRIAN BIOSKOP ===
1. Cantikya - Kursi C5
2. Andi - Kursi F1
3. Rina - Kursi B9
4. Dafa - Kursi J4
5. Aulia - Kursi D8
6. Rafi - Kursi E2
7. Zahra - Kursi G4
8. Nabila - Kursi G6
9. Fikri - Kursi I10
10. Cindy - Kursi H8
11. Bagas - Kursi C9
12. Putri - Kursi J3
13. andre - Kursi E3
14. intan - Kursi A5
15. lulu - Kursi B8
16. fafa - Kursi F2
17. Gita - Kursi H9
Pilih nomor antrian : 17
Masukkan kursi baru : I7
Kursi berhasil diubah H9 -> I7
```

Fitur Update digunakan untuk mengubah nomor kursi pembeli yang telah terdaftar dalam antrian. Pengguna memilih nomor antrian yang akan diubah, kemudian memasukkan nomor kursi baru. Sistem akan memvalidasi format nomor kursi serta memastikan kursi tersebut belum digunakan sebelum data diperbarui hingga data berhasil diupdate.

3.5 Fitur Delete

```
Pilih menu (1-10) : 4

=== PEMBELI DILAYANI ===
Nama   : Cantikya
Kursi  : C5
```

Fitur Delete digunakan untuk melayani pembeli yang berada di urutan paling depan antrian. Proses ini menerapkan operasi dequeue pada struktur data Queue sehingga pembeli

pertama yang masuk menjadi pembeli pertama yang dilayani sesuai prinsip FIFO (First In First Out).

3.6 Fitur Searching

```
Pilih menu (1-10) : 5
Masukkan nama / kursi : Zahra

=== HASIL PENCARIAN ===
6. Zahra - Kursi G4
```

Pada fitur Searching, sistem menggunakan algoritma Sequential Search untuk mencari data pembeli berdasarkan nama atau nomor kursi. Algoritma ini bekerja dengan cara menelusuri setiap node pada Linked List secara berurutan, dimulai dari node paling depan (*front*) hingga node terakhir (*rear*). Pada setiap node, sistem akan membandingkan data yang dimasukkan pengguna dengan nama atau nomor kursi yang tersimpan. Apabila data yang dicari ditemukan, sistem akan menampilkan informasi pembeli beserta posisi antriannya. Sebaliknya, apabila seluruh data telah diperiksa namun tidak ditemukan kecocokan, sistem akan menampilkan pesan "Data tidak ditemukan."

Berdasarkan hasil pengujian pada gambar di atas, sistem berhasil menemukan data yang sesuai dengan kata kunci yang dimasukkan. Hal ini menunjukkan bahwa algoritma Sequential Search telah berjalan dengan baik dalam melakukan pencarian data pembeli pada sistem antrian tiket bioskop.

3.7 Fitur Sorting

```
Pilih menu (1-10) : 6

=== SORTING NOMOR KURSI ===
1. intan - Kursi A5
2. lulu - Kursi B8
3. Rina - Kursi B9
4. Bagus - Kursi C9
5. Aulia - Kursi D8
6. Rafi - Kursi E2
7. andre - Kursi E3
8. Andi - Kursi F1
9. fafa - Kursi F2
10. Zahra - Kursi G4
11. Nabila - Kursi G6
12. Cindy - Kursi H8
13. Fikri - Kursi I10
14. Gita - Kursi I7
15. Putri - Kursi J3
16. Dafa - Kursi J4
```

Pada fitur Sorting, sistem menggunakan algoritma Merge Sort untuk mengurutkan data pembeli berdasarkan nomor kursi. Proses pengurutan diawali dengan menyalin seluruh data dari Linked List ke dalam sebuah list Python, kemudian list tersebut dibagi menjadi beberapa bagian yang lebih kecil secara rekursif. Selanjutnya, setiap bagian akan digabungkan kembali (*merge*) dengan membandingkan nilai nomor kursi sehingga menghasilkan urutan data yang teratur dari nomor kursi terkecil hingga terbesar.

Berdasarkan hasil pengujian pada gambar di atas, sistem berhasil menampilkan data pembeli yang telah diurutkan berdasarkan nomor kursi. Proses pengurutan hanya memengaruhi tampilan hasil kepada pengguna dan tidak mengubah urutan data asli pada antrian, sehingga prinsip First In First Out (FIFO) pada Queue tetap terjaga.

3.8 Penyimpanan Data (File Handling)

- Format file yang digunakan

Program menggunakan dua file berformat TXT, yaitu `data_antrian.txt` dan `data_kursi.txt`. File `data_antrian.txt` digunakan untuk menyimpan data pembeli yang terdiri

atas nama dan nomor kursi dengan format Nama|NomorKursi, sedangkan file data_kursi.txt digunakan untuk menyimpan informasi jumlah kursi yang masih tersedia. Penggunaan file TXT memungkinkan data tetap tersimpan secara permanen meskipun program ditutup.

```
data_antrian.txt
1  Andi|F1
2  Rina|B9
3  Dafa|J4
4  Aulia|D8
5  Rafi|E2
6  Zahra|G4
7  Nabila|G6
8  Fikri|I10
9  Cindy|H8
10 Bagas|C9
11 Putri|J3
12 andre|E3
13 intan|A5
14 lulu|B8
15 fafa|F2
16 Gita|I7
```

```
data_kursi.txt
1  80
```

- Cara membaca file

Saat program dijalankan, sistem akan membaca data yang tersimpan pada file data_antrian.txt dan data_kursi.txt menggunakan fungsi load_data() dan load_kursi(). Data pembeli yang terdapat pada data_antrian.txt akan dimuat ke dalam struktur data Linked List, sedangkan data pada data_kursi.txt digunakan untuk mengetahui jumlah kursi yang masih tersedia. Dengan demikian, data yang telah disimpan sebelumnya dapat langsung digunakan kembali tanpa perlu dimasukkan ulang oleh pengguna.

```
Pilih menu (1-10) : 2
```

```
=== DAFTAR ANTRIAN BIOSKOP ===
```

```
1. Andi - Kursi F1
2. Rina - Kursi B9
3. Dafa - Kursi J4
4. Aulia - Kursi D8
5. Rafi - Kursi E2
6. Zahra - Kursi G4
7. Nabila - Kursi G6
8. Fikri - Kursi I10
9. Cindy - Kursi H8
10. Bagas - Kursi C9
11. Putri - Kursi J3
12. andre - Kursi E3
13. intan - Kursi A5
14. lulu - Kursi B8
15. fafa - Kursi F2
16. Gita - Kursi I7
```

```
Pilih menu (1-10) : 9
Sisa kursi studio : 80
```

- Cara menyimpan file

Setiap kali terjadi perubahan data, seperti penambahan pembeli baru, perubahan nomor kursi, atau pelayanan antrian, sistem akan secara otomatis memperbarui isi file melalui fungsi `save_data()` dan `save_kursi()`. Dengan demikian, seluruh perubahan akan tersimpan secara permanen sehingga kondisi data tetap sama ketika program dijalankan kembali.

```
Pilih menu (1-10) : 1
Masukkan nama pembeli : Zahra
Masukkan nomor kursi : C1
Zahra berhasil masuk antrian.
```

File `data_antrian.txt` otomatis bertambah satu baris dan `data_kursi.txt` juga diperbarui.

```

≡ data_antrian.txt
1  Andi|F1
2  Rina|B9
3  Dafa|J4
4  Aulia|D8
5  Rafi|E2
6  Zahra|G4
7  Nabila|G6
8  Fikri|I10
9  Cindy|H8
10 Bagas|C9
11 Putri|J3
12 andre|E3
13 intan|A5
14 lulu|B8
15 fafa|F2
16 Gita|I7
17 Zahra|C1

```

```

≡ data_kursi.txt
1  79

```

3.9 Validasi Input dan Error Handling

Program menerapkan validasi input dan *error handling* untuk memastikan data yang dimasukkan pengguna sesuai dengan ketentuan serta mencegah terjadinya kesalahan selama program dijalankan. Validasi dilakukan pada beberapa kondisi, seperti nama pembeli, nomor kursi, nomor antrian, serta kondisi antrian. Selain itu, program juga menangani kesalahan yang mungkin terjadi saat membaca file sehingga program tetap dapat berjalan dengan baik.

Kesalahan Input	Penanganan
Nama pembeli kosong	Program menampilkan pesan "Nama tidak boleh kosong." dan proses penambahan data dibatalkan.
Nama pembeli berupa angka	Program menampilkan pesan "Nama tidak boleh angka." sehingga pengguna diminta memasukkan nama yang valid.

Nomor kursi tidak sesuai format (A1–J10)	Program menampilkan pesan "Nomor kursi tidak valid. Gunakan format A1 sampai J10." dan proses dibatalkan.
Nomor kursi sudah digunakan	Program menampilkan pesan "Nomor kursi sudah digunakan." sehingga pengguna diminta memilih nomor kursi lain.
Input nomor antrian bukan angka	Program menampilkan pesan "Input harus angka." dan proses pembaruan data dibatalkan.
Nomor antrian tidak valid	Program menampilkan pesan "Nomor tidak valid." apabila nomor antrian berada di luar jumlah data yang tersedia.
Data pencarian tidak ditemukan	Program menampilkan pesan "Data tidak ditemukan." setelah proses pencarian selesai dilakukan.
Antrian kosong	Program menampilkan pesan "Antrian kosong." ketika pengguna mencoba mengakses data yang belum tersedia.
File data tidak ditemukan	Program menggunakan mekanisme try-except sehingga program tetap dapat dijalankan tanpa mengalami <i>crash</i> .

BAB IV. PENGUJIAN

4.1 Skenario Pengujian

Pengujian dilakukan untuk memastikan setiap fitur pada sistem antrian tiket bioskop dapat berjalan sesuai dengan fungsinya. Setiap skenario diuji menggunakan data masukan yang berbeda untuk melihat apakah sistem mampu memberikan keluaran yang sesuai dengan yang diharapkan.

No	Skenario	Hasil
1	Menambahkan data pembeli baru dengan memasukkan nama dan nomor kursi yang valid.	Berhasil, data pembeli ditambahkan ke antrian, jumlah kursi berkurang, dan data tersimpan pada file.
2	Menampilkan seluruh data antrian yang telah tersimpan pada sistem.	Berhasil, seluruh data pembeli ditampilkan sesuai urutan antrian (FIFO).
3	Mengubah nomor kursi pembeli dengan memilih nomor antrian dan memasukkan nomor kursi baru yang masih tersedia.	Berhasil, nomor kursi diperbarui sesuai data yang dimasukkan.
4	Melayani pembeli yang berada pada urutan paling depan antrian (<i>dequeue</i>).	Berhasil, data pembeli pertama dihapus dari antrian dan informasi pembeli ditampilkan.
5	Mencari data pembeli berdasarkan nama atau nomor kursi menggunakan Sequential Search .	Berhasil, sistem menampilkan data yang sesuai dengan kata kunci pencarian.
6	Mengurutkan data pembeli berdasarkan nomor kursi menggunakan Merge Sort .	Berhasil, data ditampilkan dalam urutan nomor kursi yang teratur.

7	Menampilkan informasi pembeli yang berada pada posisi paling depan antrian.	Berhasil, sistem menampilkan data pembeli yang berada pada urutan pertama.
8	Menampilkan jumlah pembeli yang sedang berada di dalam antrian.	Berhasil, sistem menampilkan jumlah antrian sesuai data yang tersimpan.
9	Menampilkan informasi jumlah kursi yang masih tersedia.	Berhasil, sistem menampilkan jumlah kursi yang masih dapat dipesan.
10	Menguji validasi input, seperti nama kosong, nomor kursi tidak valid, atau nomor kursi yang telah digunakan.	Berhasil, sistem menampilkan pesan kesalahan sesuai dengan jenis input yang diberikan.

4.2 Kelebihan Sistem

Berdasarkan hasil implementasi dan pengujian, sistem memiliki beberapa kelebihan sebagai berikut:

1. Data antrian dan jumlah kursi disimpan secara permanen menggunakan file TXT, sehingga informasi tidak hilang ketika program ditutup.
2. Sistem menerapkan struktur data Queue berbasis Linked List sehingga proses pelayanan pembeli mengikuti prinsip First In First Out (FIFO) secara teratur.
3. Program menyediakan fitur CRUD (Create, Read, Update, Delete) yang memudahkan pengguna dalam mengelola data antrian.
4. Proses pencarian data menggunakan algoritma Sequential Search sehingga pengguna dapat mencari data berdasarkan nama atau nomor kursi.
5. Proses pengurutan data menggunakan algoritma Merge Sort yang memiliki performa lebih efisien dibandingkan algoritma pengurutan sederhana untuk jumlah data yang lebih besar.
6. Program dilengkapi dengan validasi input dan error handling, sehingga dapat mengurangi kesalahan pengguna saat memasukkan data.
7. Antarmuka berbasis Command Line Interface (CLI) sederhana, mudah dipahami, dan mudah dioperasikan oleh pengguna.

8. Nomor kursi yang telah digunakan tidak dapat dipilih kembali sehingga menghindari terjadinya duplikasi data pemesanan.

4.3 Keterbatasan Sistem

Walaupun sistem telah berjalan dengan baik, masih terdapat beberapa keterbatasan sebagai berikut:

1. Program masih berbasis Command Line Interface (CLI) sehingga tampilannya masih sederhana dan kurang interaktif.
2. Penyimpanan data masih menggunakan file TXT, sehingga belum mendukung pengelolaan data dalam jumlah yang sangat besar seperti database.
3. Sistem belum mendukung penggunaan oleh beberapa pengguna (*multi-user*) secara bersamaan.
4. Pengurutan nomor kursi masih berdasarkan format data yang tersimpan dan belum mempertimbangkan urutan kursi secara visual seperti denah bioskop.
5. Sistem belum menyediakan fitur login atau hak akses pengguna sehingga seluruh menu dapat diakses oleh siapa saja.
6. Belum tersedia fitur pembatalan pemesanan (*cancel booking*) maupun pencetakan tiket secara otomatis.

BAB V. KENDALA DAN SOLUSI

Selama proses perancangan dan pengembangan sistem antrian tiket bioskop, terdapat beberapa kendala yang dihadapi. Kendala tersebut berkaitan dengan implementasi struktur data, validasi input, penyimpanan data, serta pengelolaan nomor kursi. Berbagai solusi diterapkan agar sistem dapat berjalan dengan baik dan sesuai dengan kebutuhan pengguna.

Kendala	Solusi
Data antrian tidak tersimpan setelah program ditutup.	Menerapkan file handling menggunakan file TXT (data_antrian.txt dan data_kursi.txt) agar data tersimpan secara permanen.
Nomor kursi yang sama dapat dipilih oleh lebih dari satu pembeli.	Menambahkan proses pengecekan nomor kursi sebelum data disimpan sehingga hanya kursi yang masih tersedia yang dapat dipilih.
Kesalahan input nama atau nomor kursi menyebabkan data tidak valid.	Menambahkan validasi input untuk memastikan nama tidak kosong, bukan angka, serta nomor kursi sesuai format A1–J10.
Pengelolaan urutan pelayanan pembeli harus sesuai dengan urutan kedatangan.	Menggunakan struktur data Queue berbasis Linked List agar pelayanan mengikuti prinsip First In First Out (FIFO).
Data pembeli sulit dicari ketika jumlah antrian bertambah.	Mengimplementasikan algoritma Sequential Search untuk mempermudah pencarian berdasarkan nama atau nomor kursi.
Data nomor kursi perlu ditampilkan secara terurut.	Mengimplementasikan algoritma Merge Sort untuk mengurutkan data berdasarkan nomor kursi sebelum ditampilkan kepada pengguna.

Selama proses pengerjaan proyek Sistem Antrian Tiket Bioskop, penulis menghadapi beberapa tantangan dalam mengimplementasikan struktur data Queue

berbasis Linked List agar seluruh fitur dapat berjalan dengan baik. Tantangan utama yang dihadapi adalah mengatur hubungan antar node, memastikan proses *enqueue* dan *dequeue* berjalan sesuai prinsip First In First Out (FIFO), serta mengintegrasikan fitur pencarian, pengurutan, dan pembaruan data tanpa mengganggu struktur antrian yang sudah ada.

Selain itu, penulis juga menghadapi tantangan dalam menerapkan file handling agar data antrian dan jumlah kursi tetap tersimpan setelah program ditutup. Proses pembuatan validasi input, seperti pengecekan format nomor kursi, nama pembeli, dan ketersediaan kursi, juga memerlukan beberapa kali pengujian untuk memastikan program dapat menangani berbagai kondisi dengan baik. Melalui proses tersebut, penulis memperoleh pemahaman yang lebih mendalam mengenai penerapan algoritma, struktur data, serta pentingnya pengujian dalam pengembangan perangkat lunak.

BAB VI. PEMBAGIAN TUGAS

Pembagian tugas dilakukan untuk mempermudah proses pengembangan sistem dan memastikan setiap anggota berkontribusi pada fitur yang berbeda. Seluruh anggota juga terlibat dalam proses diskusi, pengujian program, serta penyusunan laporan akhir.

Nama	Tugas dan Kontribusi
Intan Sahara Nurafni	Bertanggung jawab dalam perancangan dan implementasi struktur data utama yang digunakan pada sistem, yaitu Linked List dan Queue. Mengembangkan kelas Node dan QueueBioskop, mengatur proses enqueue dan dequeue, mengelola pointer front dan rear, serta memastikan sistem antrian berjalan sesuai prinsip First In First Out (FIFO). Selain itu, turut membantu pengujian fungsi antrian dan integrasi dengan fitur lainnya.
Naylatul Fadhilah	Bertanggung jawab dalam implementasi fitur CRUD (Create, Read, Update, Delete) dan File Handling. Selain itu, merancang dan membuat mekanisme penyimpanan data menggunakan file data_antrian.txt dan data_kursi.txt, mengimplementasikan fungsi pembacaan data (load_data() dan load_kursi()) serta penyimpanan data (save_data() dan save_kursi()), sehingga seluruh data antrian dan informasi kursi tetap tersimpan secara permanen meskipun program ditutup dan dijalankan kembali.
Faiza Aghnaita Zahra	Bertanggung jawab dalam implementasi algoritma Searching, Sorting, dan Validasi Input. Mengembangkan fitur pencarian data menggunakan Sequential Search, pengurutan data menggunakan Merge Sort, validasi nama pembeli dan nomor kursi, serta penanganan berbagai kesalahan input (error handling) agar program dapat berjalan dengan lebih aman dan stabil.

Selain tugas utama masing-masing, seluruh anggota kelompok berpartisipasi dalam perancangan flowchart, pengujian sistem, perbaikan bug (debugging), dokumentasi program, dan penyusunan laporan akhir proyek serta pembuatan video.

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Program Sistem Antrian Tiket Bioskop berhasil dikembangkan menggunakan bahasa pemrograman Python dengan menerapkan struktur data Queue berbasis Linked List, sehingga proses pelayanan pembeli dapat berjalan sesuai prinsip First In First Out (FIFO).
2. Seluruh fitur utama pada sistem, seperti Create, Read, Update, Delete (CRUD), Searching menggunakan algoritma Sequential Search, Sorting menggunakan algoritma Merge Sort, serta file handling, telah berhasil diimplementasikan dan berfungsi sesuai dengan kebutuhan sistem.
3. Program dilengkapi dengan validasi input dan penyimpanan data secara permanen menggunakan file TXT, sehingga mampu meminimalkan kesalahan pengguna serta menjaga data antrian dan informasi jumlah kursi tetap tersimpan meskipun program ditutup.

Saran Pengembangan

Untuk pengembangan pada masa mendatang, terdapat beberapa fitur yang dapat ditambahkan agar sistem menjadi lebih optimal, yaitu:

1. Mengembangkan antarmuka program menjadi berbasis Graphical User Interface (GUI) atau website sehingga lebih menarik dan mudah digunakan oleh pengguna.
2. Mengganti media penyimpanan dari file TXT ke database agar pengelolaan data menjadi lebih aman, efisien, dan mampu menangani jumlah data yang lebih besar.

3. Menambahkan fitur pemilihan kursi dalam bentuk denah bioskop, sistem login pengguna, pembatalan pemesanan tiket, serta pencetakan tiket secara otomatis untuk meningkatkan fungsionalitas sistem.

DAFTAR PUSTAKA

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.
3. Miller, B., & Ranum, D. (2011). *Problem Solving with Algorithms and Data Structures Using Python*. Franklin, Beedle & Associates.

LAMPIRAN

Lampiran A. Link Github

https://github.com/intnshr/ASD_A_14

Lampiran B. Screenshot Program

Tampilan Menu Utama

```
=== SISTEM ANTRIAN TIKET BIOSKOP ===
1. Tambah Antrian
2. Tampilkan Antrian
3. Update Nomor Kursi
4. Layani Antrian
5. Cari Data
6. Sorting Nomor Kursi
7. Lihat Antrian Depan
8. Jumlah Antrian
9. Info Kursi
10. Keluar
Pilih menu (1-10) : █
```

Fitur Tambah Antrian

```
Pilih menu (1-10) : 1
Masukkan nama pembeli : nana
Masukkan nomor kursi : d7
nana berhasil masuk antrian.
```

Fitur Tampilkan Antrian

```
Pilih menu (1-10) : 2

=== DAFTAR ANTRIAN BIOSKOP ===
1. Nayla - Kursi A9
2. Faiza - Kursi H9
3. Cantikya - Kursi C5
4. Andi - Kursi F1
5. Rina - Kursi B9
6. Dafa - Kursi J4
7. Aulia - Kursi D8
8. Rafi - Kursi E2
9. Zahra - Kursi G4
10. Nabila - Kursi G6
11. Fikri - Kursi I10
12. Cindy - Kursi H8
13. Bagas - Kursi C9
14. Putri - Kursi J3
15. vivi - Kursi J9
16. nana - Kursi D7
```

Fitur Update Nomor Kursi

```
Pilih menu (1-10) : 3

=== DAFTAR ANTRIAN BIOSKOP ===
1. Nayla - Kursi A9
2. Faiza - Kursi H9
3. Cantikya - Kursi C5
4. Andi - Kursi F1
5. Rina - Kursi B9
6. Dafa - Kursi J4
7. Aulia - Kursi D8
8. Rafi - Kursi E2
9. Zahra - Kursi G4
10. Nabila - Kursi G6
11. Fikri - Kursi I10
12. Cindy - Kursi H8
13. Bagas - Kursi C9
14. Putri - Kursi J3
15. vivi - Kursi J9
16. nana - Kursi D7
Pilih nomor antrian : 5
Masukkan kursi baru : b3
Kursi berhasil diubah B9 -> B3
```

Fitur Layani Antrian

```
Pilih menu (1-10) : 4

=== PEMBELI DILAYANI ===
Nama : Nayla
Kursi : A9
```

Fitur Cari Data (Searching)

```
Pilih menu (1-10) : 5
Masukkan nama / kursi : vivi

=== HASIL PENCARIAN ===
14. vivi - Kursi J9
```

Fitur Sorting Nomor Kursi

```
Pilih menu (1-10) : 6

=== SORTING NOMOR KURSI ===
1. Rina - Kursi B3
2. Cantikya - Kursi C5
3. Bagas - Kursi C9
4. nana - Kursi D7
5. Aulia - Kursi D8
6. Rafi - Kursi E2
7. Andi - Kursi F1
8. Zahra - Kursi G4
9. Nabila - Kursi G6
10. Cindy - Kursi H8
11. Faiza - Kursi H9
12. Fikri - Kursi I10
13. Putri - Kursi J3
14. Dafa - Kursi J4
15. vivi - Kursi J9
```

Fitur Lihat Antrian Depan

```
Pilih menu (1-10) : 7

=== ANTRIAN PALING DEPAN ===
Nama    : Faiza
Kursi    : H9
```

Fitur Jumlah Antrian

```
Pilih menu (1-10) : 8
Jumlah antrian saat ini : 15
```

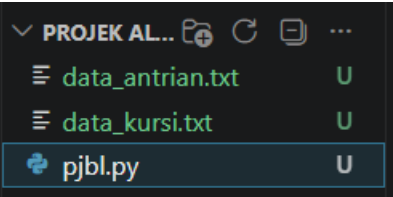
Fitur Info Kursi


```
Pilih menu (1-10) : 9
Sisa kursi studio : 84
```

Fitur Keluar

```
Pilih menu (1-10) : 10
Program selesai.
```

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History Github

Commits			
History for ASD_A_14 / projek.py on main		All users	All time
Commits on Jun 2, 2026			
Update projek.py	intnsr authored 3 weeks ago	Verified 13aaa58	📄 📁 🔍 <>
Update projek.py	naylatul79 authored 3 weeks ago	Verified b1f9e0a	📄 📁 🔍 <>
Update projek.py	payja-pixel authored 3 weeks ago	Verified 3a228da	📄 📁 🔍 <>
Update projek.py	naylatul79 authored 3 weeks ago	Verified 3aa7beb	📄 📁 🔍 <>
Refactor and enhance ticket queue system	intnsr authored 3 weeks ago	Verified dc97df8	📄 📁 🔍 <>
Restore original content and functionality of projek.py	payja-pixel authored 3 weeks ago	Verified 35bdb4d	📄 📁 🔍 <>
Update projek.py	naylatul79 authored 3 weeks ago	Verified 9f6e090	📄 📁 🔍 <>
Update projek.py	intnsr authored 3 weeks ago	Verified d95333b	📄 📁 🔍 <>
Update projek.py	intnsr authored 3 weeks ago	Verified 67877b5	📄 📁 🔍 <>
Update projek.py	intnsr authored 3 weeks ago	Verified 3fd98d8	📄 📁 🔍 <>
Update projek.py	intnsr authored 3 weeks ago	Verified b20ab28	📄 📁 🔍 <>
Update projek.py	intnsr authored 3 weeks ago	Verified cd31b15	📄 📁 🔍 <>

Lampiran E. Source Code Utama

```
pjbl.py > ...
1  # =====
2  # PROJECT PJBL
3  # SISTEM ANTRIAN TIKET BIOSKOP
4  # KELOMPOK : THREECUTE
5  # =====
6
7  import os
8
9  # =====
10 # CLASS NODE (LINKED LIST)
11 # =====
12
13 class Node:
14
15     def __init__(self, nama, kursi):
16
17         self.nama = nama
18         self.kursi = kursi
19         self.next = None
20
21
22 # =====
23 # CLASS QUEUE BIOSKOP
24 # =====
25
26 class QueueBioskop:
27
28     def validasi_kursi(self, kursi):
29
30         if len(kursi) < 2:
31             return False
32
33         huruf = kursi[0]
34         angka = kursi[1:]
35
36         if huruf not in "ABCDEFGHIJ":
37             return False
```

```
26 class QueueBioskop:
27     def validasi_kursi(self, kursi):
38
39         if not angka.isdigit():
40             return False
41
42         angka = int(angka)
43
44         if angka < 1 or angka > 10:
45             return False
46
47         return True
48
49     def __init__(self):
50
51         # Pointer queue
52         self.front = None
53         self.rear = None
54
55         # Jumlah antrian
56         self.jumlah = 0
57
58         # Total kursi studio
59         self.total_kursi = 100
60
61         # Lokasi file
62         BASE_DIR = os.path.dirname(os.path.abspath(__file__))
63
64         self.file_antrian = os.path.join(
65             BASE_DIR,
66             "data_antrian.txt"
67         )
68
69         self.file_kursi = os.path.join(
70             BASE_DIR,
71             "data_kursi.txt"
72         )
```

```

26 class QueueBioskop:
49     def __init__(self):
73
74         # Load data dari txt
75         self.load_data()
76         self.load_kursi()
77
78         # =====
79         # LOAD DATA ANTRIAN
80         # =====
81
82     def load_data(self):
83
84         try:
85
86             with open(
87                 self.file_antrian,
88                 "r",
89                 encoding="utf-8"
90             ) as file:
91
92                 for baris in file:
93
94                     data = baris.strip().split("|")
95
96                     if len(data) == 2:
97
98                         nama = data[0]
99                         kursi = data[1]
100
101                         self.tambah_node(nama, kursi)
102
103         except FileNotFoundError:
104
105             print("File antrian tidak ditemukan.")
106
107         # =====

```

```

26  class QueueBioskop:
107      # =====
108      # LOAD DATA KURSI
109      # =====
110
111      def load_kursi(self):
112
113          try:
114
115              with open(self.file_kursi, "r") as file:
116
117                  self.kursi_tersedia = int(
118                      file.read()
119                  )
120
121          except:
122
123              self.kursi_tersedia = self.total_kursi
124
125      # =====
126      # SAVE DATA ANTRIAN
127      # =====
128
129      def save_data(self):
130
131          with open(
132              self.file_antrian,
133              "w",
134              encoding="utf-8"
135          ) as file:
136
137              temp = self.front
138
139              while temp:
140
141                  file.write(
142                      f"{temp.nama}||{temp.kursi}\n"

```

```

26  class QueueBioskop:
129      def save_data(self):
142          f"{temp.nama}|{temp.kursi}\n"
143          )
144
145          temp = temp.next
146
147      # =====
148      # SAVE DATA KURSI
149      # =====
150
151      def save_kursi(self):
152
153          with open(self.file_kursi, "w") as file:
154
155              file.write(
156                  str(self.kursi_tersedia)
157              )
158
159      # =====
160      # TAMBAH NODE
161      # =====
162
163      def tambah_node(self, nama, kursi):
164
165          node_baru = Node(nama, kursi)
166
167          # Jika antrian kosong
168          if self.front is None:
169
170              self.front = node_baru
171              self.rear = node_baru
172
173          else:
174
175              self.rear.next = node_baru
176              self.rear = node_baru

```

```

26     class QueueBioskop:
163         def tambah_node(self, nama, kursi):
178             self.jumlah += 1
179
180             # =====
181             # CEK KURSI
182             # =====
183
184         def cek_kursi(self, kursi):
185
186             temp = self.front
187
188             while temp:
189
190                 if temp.kursi == kursi:
191
192                     return True
193
194                 temp = temp.next
195
196             return False
197
198             # =====
199             # ENQUEUE
200             # =====
201
202         def enqueue(self):
203
204             # Jika kursi habis
205             if self.kursi_tersedia == 0:
206
207                 print("\n=====")
208                 print("KURSI BIOSKOP SUDAH HABIS")
209                 print("ANTRIAN DITUTUP")
210                 print("=====")
211

```

```

26     class QueueBioskop:
202         def enqueue(self):
212             return
213
214             nama = input(
215                 "Masukkan nama pembeli : "
216             ).strip()
217
218             # Validasi nama
219             if nama == "":
220
221                 print("Nama tidak boleh kosong.")
222                 return
223
224             if nama.isdigit():
225
226                 print("Nama tidak boleh angka.")
227                 return
228
229             kursi = input(
230                 "Masukkan nomor kursi : "
231             ).upper().strip()
232
233             if not self.validasi_kursi(kursi):
234
235                 print(
236                     "Nomor kursi tidak valid.\n"
237                     "Gunakan format A1 sampai J10."
238                 )
239
240                 return
241
242
243             # Validasi kursi
244             if kursi == "":
245
246                 print("Nomor kursi tidak boleh kosong.")

```



```

26  class QueueBioskop:
202      def enqueue(self):
247          return
248
249          # Cek kursi sudah dipakai
250          if self.cek_kursi(kursi):
251              print("Nomor kursi sudah digunakan.")
252              return
253
254          # Tambah node
255          self.tambah_node(nama, kursi)
256
257          # Kursi berkurang
258          self.kursi_tersedia -= 1
259
260          # Save file
261          self.save_data()
262          self.save_kursi()
263
264          print(f"{nama} berhasil masuk antrian.")
265
266          # =====
267          # DEQUEUE
268          # =====
269
270
271      def dequeue(self):
272
273          if self.front is None:
274              print("Antrian kosong.")
275              return
276
277          nama = self.front.nama
278          kursi = self.front.kursi
279
280
281          # Geser front

```

```

26  class QueueBioskop:
271      def dequeue(self):
281          # Geser front
282          self.front = self.front.next
283
284          # Jika kosong
285          if self.front is None:
286              self.rear = None
287
288
289          self.jumlah -= 1
290
291          # Save data
292          self.save_data()
293
294          print("\n=== PEMBELI DILAYANI ===")
295          print(f>Nama      : {nama}")
296          print(f>Kursi    : {kursi}")
297
298          # =====
299          # TAMPILKAN ANTRIAN
300          # =====
301
302      def tampilkan_antrian(self):
303
304          if self.front is None:
305
306              print("Antrian kosong.")
307              return
308
309          print("\n=== DAFTAR ANTRIAN BIOSKOP ===")
310
311          temp = self.front
312          nomor = 1
313
314          while temp:
315

```

```

26     class QueueBioskop:
302         def tampilkan_antrian(self):
316             print(
317                 f"{nomor}. "
318                 f"{temp.nama} - Kursi {temp.kursi}"
319             )
320
321             temp = temp.next
322             nomor += 1
323
324             # =====
325             # UPDATE NOMOR KURSI
326             # =====
327
328         def update_kursi(self):
329
330             if self.front is None:
331
332                 print("Antrian kosong.")
333                 return
334
335             self.tampilkan_antrian()
336
337             try:
338
339                 nomor = int(
340                     input("Pilih nomor antrian : ")
341                 )
342
343             except ValueError:
344
345                 print("Input harus angka.")
346                 return
347
348             if nomor < 1 or nomor > self.jumlah:
349
350                 print("Nomor tidak valid.")

```

```

26     class QueueBioskop:
328         def update_kursi(self):
351             return
352
353             kursi_baru = input(
354                 "Masukkan kursi baru : "
355             ).upper().strip()
356
357             if not self.validasi_kursi(kursi_baru):
358
359                 print(
360                     "Nomor kursi tidak valid.\n"
361                     "Gunakan format A1 sampai J10."
362                 )
363
364                 return
365
366             # Validasi
367             if kursi_baru == "":
368
369                 print("Nomor kursi tidak boleh kosong.")
370                 return
371
372             # Cek kursi
373             if self.cek_kursi(kursi_baru):
374
375                 print("Nomor kursi sudah digunakan.")
376                 return
377
378             # Traversal linked list
379             temp = self.front
380
381             for i in range(1, nomor):
382
383                 temp = temp.next
384
385             kursi_lama = temp.kursi

```

```

26  class QueueBioskop:
328      def update_kursi(self):
386
387          temp.kursi = kursi_baru
388
389          # Save data
390          self.save_data()
391
392          print(
393              f"Kursi berhasil diubah "
394              f"{kursi_lama} -> {kursi_baru}"
395          )
396
397          # =====
398          # SEARCHING
399          # =====
400
401      def cari_data(self):
402
403          if self.front is None:
404
405              print("Antrian kosong.")
406              return
407
408          keyword = input(
409              "Masukkan nama / kursi : "
410          ).lower()
411
412          temp = self.front
413          nomor = 1
414          ditemukan = False
415
416          print("\n=== HASIL PENCARIAN ===")
417
418          while temp:
419
420              if (

```

```

26     class QueueBioskop:
401         def cari_data(self):
421             keyword in temp.nama.lower()
422             or keyword in temp.kursi.lower()
423         ):
424
425             print(
426                 f"{nomor}. "
427                 f"{temp.nama} - Kursi {temp.kursi}"
428             )
429
430             ditemukan = True
431
432             temp = temp.next
433             nomor += 1
434
435         if not ditemukan:
436
437             print("Data tidak ditemukan.")
438
439         # =====
440         # SORTING KURSI
441         # =====
442         def merge_sort(self, data):
443
444             if len(data) <= 1:
445                 return data
446
447             tengah = len(data) // 2
448
449             kiri = self.merge_sort(data[:tengah])
450             kanan = self.merge_sort(data[tengah:])
451
452             hasil = []
453
454             i = 0
455             j = 0

```

```
26  class QueueBioskop:
442      def merge_sort(self, data):
457          while i < len(kiri) and j < len(kanan):
458              if kiri[i][1] <= kanan[j][1]:
459                  hasil.append(kiri[i])
460                  i += 1
461              else:
462                  hasil.append(kanan[j])
463                  j += 1
464              while i < len (kiri):
465                  hasil.append(kiri[i])
466                  i += 1
467              while j < len(kanan):
468                  hasil.append(kanan[j])
469                  j += 1
470              return hasil
471
472      def sorting_kursi(self):
473          if self.front is None:
474              print("Antrian kosong.")
475              return
476          data = []
477          temp = self.front
478
479
480
481
482
483
484
485
486
487
488
489
490
491
```

```

26     class QueueBioskop:
481         def sorting_kursi(self):
492             while temp:
493                 data.append(
494                     (temp.nama, temp.kursi)
495                 )
496                 temp = temp.next
497
498             # Merge Sort
499             data = self.merge_sort(data)
500
501             print("\n=== SORTING NOMOR KURSI ===")
502
503             for i, item in enumerate(data, start=1):
504                 print(
505                     f"{i}. "
506                     f"{item[0]} - Kursi {item[1]}"
507                 )
508
509             # =====
510             # LIHAT ANTRIAN DEPAN
511             # =====
512
513         def lihat_depan(self):
514             if self.front is None:
515                 print("Antrian kosong.")
516                 return
517
518             print("\n=== ANTRIAN PALING DEPAN ===")
519             print(f>Nama      : {self.front.nama}")
520             print(f>Kursi    : {self.front.kursi}")
521
522

```



```

26  class QueueBioskop:
527      # =====
528      # JUMLAH ANTRIAN
529      # =====
530
531      def jumlah_antrian(self):
532
533          print(
534              f"Jumlah antrian saat ini : "
535              f"{self.jumlah}"
536          )
537
538      # =====
539      # INFO KURSI
540      # =====
541
542      def info_kursi(self):
543
544          print(
545              f"Sisa kursi studio : "
546              f"{self.kursi_tersedia}"
547          )
548
549      # =====
550      # MENU PROGRAM
551      # =====
552
553      def menu(self):
554
555          while True:
556
557              print("\n=== SISTEM ANTRIAN TIKET BIOSKOP ===")
558              print("1. Tambah Antrian")
559              print("2. Tampilkan Antrian")
560              print("3. Update Nomor Kursi")
561              print("4. Layani Antrian")
562              print("5. Cari Data")

```

```
26     class QueueBioskop:
553         def menu(self):
562             print("5. Cari Data")
563             print("6. Sorting Nomor Kursi")
564             print("7. Lihat Antrian Depan")
565             print("8. Jumlah Antrian")
566             print("9. Info Kursi")
567             print("10. Keluar")
568
569             pilih = input(
570                 "Pilih menu (1-10) : "
571             ).strip()
572
573             if pilih == "1":
574                 self.enqueue()
575
576             elif pilih == "2":
577                 self.tampilkan_antrian()
578
579             elif pilih == "3":
580                 self.update_kursi()
581
582             elif pilih == "4":
583                 self.dequeue()
584
585             elif pilih == "5":
586                 self.cari_data()
587
588             elif pilih == "6":
589                 self.sorting_kursi()
590
591             elif pilih == "7":
592                 self.lihat_antrian_depan()
593
594             elif pilih == "8":
595                 self.jumlah_antrian()
596             elif pilih == "9":
597                 self.info_kursi()
598             elif pilih == "10":
599                 self.keluar()
600             else:
601                 print("Pilihan tidak valid. Silakan pilih menu yang tersedia.")
```

```
26  class QueueBioskop:
553      def menu(self):
597          elif pilih == "7":
598              |
599              self.lihat_depan()
600
601          elif pilih == "8":
602              |
603              self.jumlah_antrian()
604
605          elif pilih == "9":
606              |
607              self.info_kursi()
608
609          elif pilih == "10":
610              |
611              print("Program selesai.")
612              break
613
614          else:
615              |
616              print("Pilihan tidak valid.")
617
618      # =====
619      # PROGRAM UTAMA
620      # =====
621
622      bioskop = QueueBioskop()
623      bioskop.menu()
```